

Assignment 4 COSC 2347.

DUE 11:59 pm on 3/31/25 @ 11:59 pm. No late assignments.

Changes are highlighted in yellow.

Using correct C# style and naming conventions as detailed in the lectures and slides, create a C# OOP program as described below.

Zoo Keeper

You were hired by the local zoo to model their operations. I will provide you a test file of “zoo events” and you will read that file into a CommandQueue array or queue or list (using file operations, not console redirection) and you will process the queue, producing console output as detailed below. Do not read an item from the file and then process it; read them into a queue or list or array and then when complete, process them.

You will build an array of objects representing the actual live animals in the zoo: Monkey, Lion, Alligator, Predator (like the movie). Each time an animal is born, it will be added to the array and its Birthday will be set to the current day. When an animal dies, it will remain in the array but its Deathdate will be set to the current day. Future events don't affect dead animals.

The zoo classifies their animals as mammals, reptiles, or extraterrestrials.

The zoo keeps records of every animal and tracks classification and the following information: day of birth, day of death, favorite food, and their strength. Animal information:

Animal	Classification	Favorite Food	Strength
Monkey	Mammal	Banana	100
Lion	Mammal	Deer	200
Alligator	Reptile	Snakes	250
Predator	Extraterrestrial	Everything	10000

List of events:

The event syntax is indented and listed below.

Whenever these events happen, they circulate to all of the animals in the zoo, and they react accordingly:

Feeding

Plague 'animal classification'

Sunrise

Special note: When it is sunrise, it's a new day and you should increase your day counter and say that it's a new day. Before the new day output is the only place where you can add a blank line.

Special note: When the Predator feeds, it eats everything else in the zoo. All other animals eat their favorite food. Make sure you print what food each animal is eating and all of the animals that the predator eats.

Special note: Plague only affects items of a specified AnimalClassification, and it kills everything that it affects. Make sure you print that plague broke out and what animals are killed by the plague.

Special note: When it's feeding time, make sure you print out all of the animals that are feeding and what they're eating.

When these events happen, it affects the first live animals of that type:

Fight animal1 animal2

Death animal

Special note: when 2 animals fight, the stronger one kills the weaker one. Make sure you print who fought and what animal was killed.

Special note: When an animal dies by command, print out the animal that has died from natural causes.

You can also have:

Birth animal

Report

Special note: Report = print status for all animals you've tracked. In order, print what the animal type is, its classification, its favorite food, its strength, and its birth and death days (0 death day if it's still alive).

Special note: Birth animal = make sure you print what animal has been born.

Notes:

1. This program must be solved using OOP and reasonable class designs. The point of this program is to exercise the information we've learned in chapters 6 and 7. If you solve the program using structured programming, you will receive 0 points.
2. The input data is guaranteed to be valid, you don't have to error check it and I will not ask you to do invalid things.
3. Output the program's results to the console using the standard `Console.Write` and `Console.WriteLine` commands. **DO NOT ALLOW YOUR OUTPUT TO EXCEED 1 SCREEN so that you can capture all of it! Double check that no output has scrolled off. Each event can only take up 1 line on the screen per animal. Only include blank lines when told that it's a new day.**
4. Your program cannot throw exceptions from conversion results.
5. Capture the output screen with the correct results. If your output screen is disappearing before you can capture it, put in a `ReadLine` statement at the end of the program and then you can capture the output window. Now that we're not doing console redirection, the `ReadLine` statements will do what you expect and stop execution and wait.
6. Turn your complete source code along with the screen capture of your complete program output into Blackboard by the assignment due date. Failure to turn in full source code (in text format) and screen output snapshots showing all of your program output = grade of 0.
7. Like all assignments in this class, this is NOT a group project. Writing your code with someone else or sharing code is academic dishonesty and will be dealt with accordingly.
8. As explained in the syllabus, these instructions are subject to verbal change in this class. If you are not paying attention or not here and miss any changes in instructions, that is on you.

File I/O code to read the data file:

```
// Create a FileStream and put your input file in a location
// that matches whatever path that you specify below.
FileStream stream = new FileStream("c:\\temp\\AS4 input2.txt", FileMode.Open);

// Now create a streamreader and pass it the filestream you created above
StreamReader reader = new StreamReader(stream);
string inputString;

//This will read 1 line from the file
inputString = reader.ReadLine();

// When you are done reading from the file (you run out
// of lines to read and inputString == null), you should:
reader.Dispose();
```